**MINI-ASSESSMENT**

**MULTI-STEP PROMPT WORKFLOW**

**Transforming Raw API Notes into a Polished Technical Documentation Guide**

**STUDENT DETAILS**

**Name:**SHAIK THANVIR AHMED

**Register Number:** 99230040768

**Date:** 7/9/2026

## 1. INTRODUCTION

This mini-assessment demonstrates the design and execution of a multi-step prompt workflow using an AI interface.

The selected task is to transform unstructured project notes about a fictional Student Management REST API into professional technical documentation.

The workflow consists of three connected stages:

**Step 1:** Extraction and Planning

**Step 2:** Generation and Drafting

**Step 3:** Evaluation and Refinement

The output of each stage is used as the input or basis for the next stage. Therefore, the workflow demonstrates prompt chaining rather than a single-shot interaction.

## 2. SCENARIO SELECTED

**Technical Documentation**

The scenario selected for this assessment is **Technical Documentation**.

The task begins with messy project notes describing a Student Management API. The AI first analyzes these notes and creates a structured documentation plan. The plan is then transformed into a complete API guide. Finally, the generated guide is reviewed against a specific checklist and refined into a final version.

**Overall Workflow**

**Raw Notes → Structured Plan → API Documentation Draft → Critique → Final Polished API Guide**

## 3. OBJECTIVE

The main objectives of the workflow are:

1. Extract important information from unstructured technical notes.

2. Organize the extracted information into a logical documentation plan.
3. Generate a developer-facing API documentation draft.
4. Evaluate the draft against specific quality requirements.
5. Correct identified issues and produce a final polished document.
6. Demonstrate dependency between multiple prompts.

**4. RAW PROJECT NOTES**

The following information is used as the initial input for the workflow.

The Student Management API allows applications to manage student records.

Users authenticate using an API key.

The API supports creating, viewing, updating, and deleting students.

A student has:

- ID
- Name
- Email
- Department
- Year

The API supports the following endpoints:

- GET /students — returns all students.
- GET /students/{id} — returns one student.
- POST /students — creates a student.
- PUT /students/{id} — updates a student.
- DELETE /students/{id} — deletes a student.

Successful requests return JSON.

Invalid requests may return HTTP 400.

Unauthorized requests return HTTP 401.

A student that does not exist returns HTTP 404.

The API should validate email addresses.

Applications should never expose API keys publicly.

Clients should handle errors gracefully.

**5. STEP 1 — EXTRACTION AND PLANNING**

### 5.1 Purpose

The purpose of Step 1 is to analyze the raw project notes and convert the unstructured information into a structured documentation plan.

The AI is instructed not to write the complete documentation at this stage.

### 5.2 Prompt 1

Act as a senior technical documentation architect.

I am going to provide messy notes about a fictional REST API for a Student Management System.

Your task is to analyze the notes and create a structured outline for an API documentation guide.

The outline must contain:

1. API overview
2. Authentication
3. Main endpoints
4. Request parameters
5. Request/response examples
6. Error handling
7. Security considerations
8. Common usage workflow

Identify the most important technical information and organize it logically.

Do not write the full documentation yet. Only produce the structured documentation plan.

Here are the raw project notes:

The Student Management API allows applications to manage student records.

Users authenticate using an API key.

The API supports creating, viewing, updating, and deleting students.

A student has an ID, name, email, department, and year.

GET /students returns all students.

GET /students/{id} returns one student.

POST /students creates a student.

PUT /students/{id} updates a student.

DELETE /students/{id} deletes a student.

Successful requests return JSON.

Invalid requests may return 400.

Unauthorized requests return 401.

A student that does not exist returns 404.

The API should validate email addresses.

Applications should never expose API keys publicly.

Clients should handle errors gracefully.

**5.3 Step 1 — AI Response**

**Student Management API — Documentation Plan**

**1. API Overview**

- Purpose of the Student Management API
- Main capabilities
- Student information managed by the API
- JSON response format

**2. Authentication**

- API-key authentication
- How authentication is used
- Importance of protecting API keys

**3. Main Endpoints**

- GET /students — retrieve all students
- GET /students/{id} — retrieve a specific student
- POST /students — create a new student
- PUT /students/{id} — update a student
- DELETE /students/{id} — delete a student

**4. Request Parameters**

- Student ID
- Student name
- Email
- Department
- Year
- Parameters associated with individual endpoints

**5. Request/Response Examples**

- Example requests
- Example responses

- JSON formatting
- Examples for major API operations

**6. Error Handling**

- HTTP 400 — invalid request
- HTTP 401 — unauthorized request
- HTTP 404 — student not found
- General error-handling guidance

**7. Security Considerations**

- Protect API keys
- Never expose API keys publicly
- Validate input data such as email addresses

**8. Common Usage Workflow**

- Authenticate using the API key
- Retrieve student records
- Retrieve a specific student
- Create a student
- Update a student
- Delete a student
- Handle errors appropriately

**INSERT IMAGE HERE**

**Figure 1: Step 1 – Extraction and Planning showing Prompt 1 and the structured AI-generated documentation plan.**

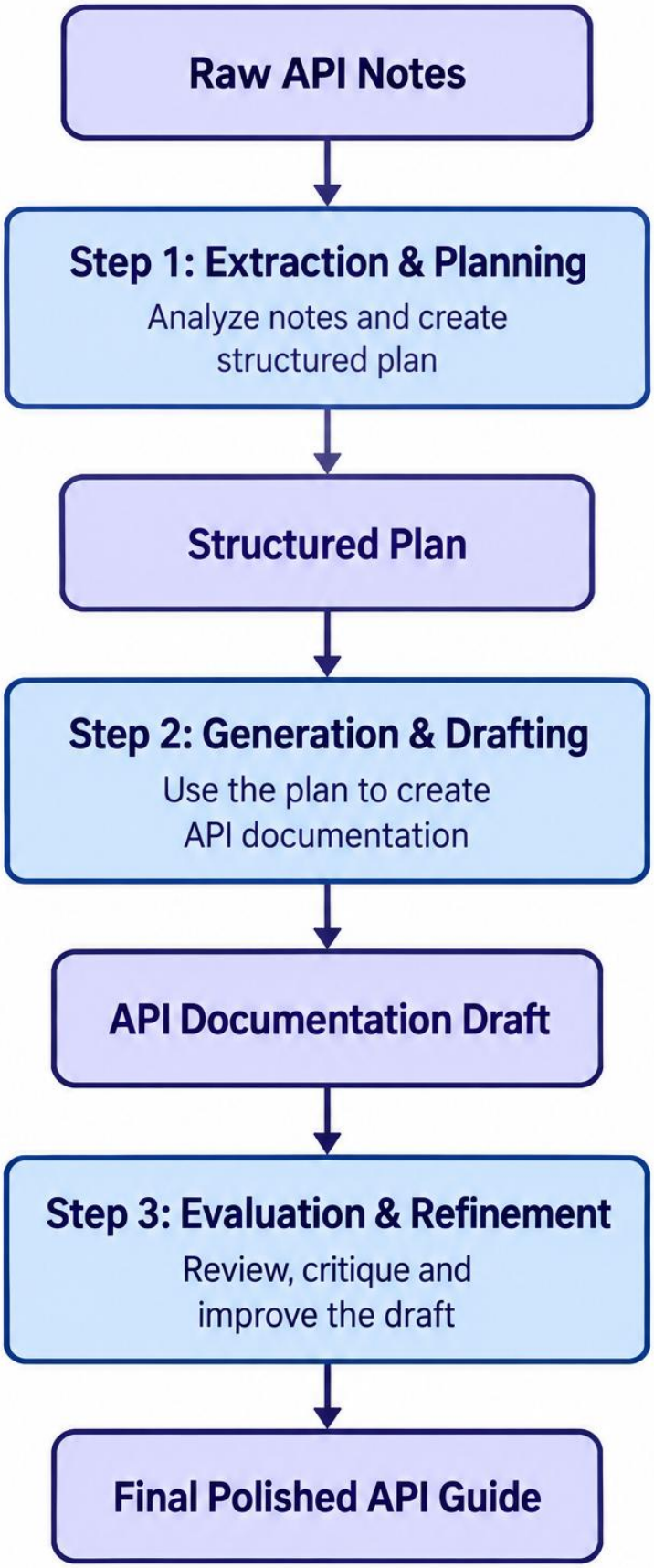
## 6. STEP 2 — GENERATION AND DRAFTING

### 6.1 Purpose

The purpose of Step 2 is to use the structured documentation plan generated in Step 1 to create the first complete API documentation draft.

This establishes a direct dependency:

**Step 1 Output → Step 2 Input**


Raw API Notes
Step 1: Extraction & Planning
Analyze notes and create structured plan
Structured Plan
Step 2: Generation & Drafting
Use the plan to create API documentation
API Documentation Draft
Step 3: Evaluation & Refinement
Review, critique and improve the draft
Final Polished API Guide

### 6.2 Prompt 2

Excellent. Now use ONLY the structured documentation plan you just created in Step 1 as the basis for generating the actual API documentation.

Act as a senior technical writer.

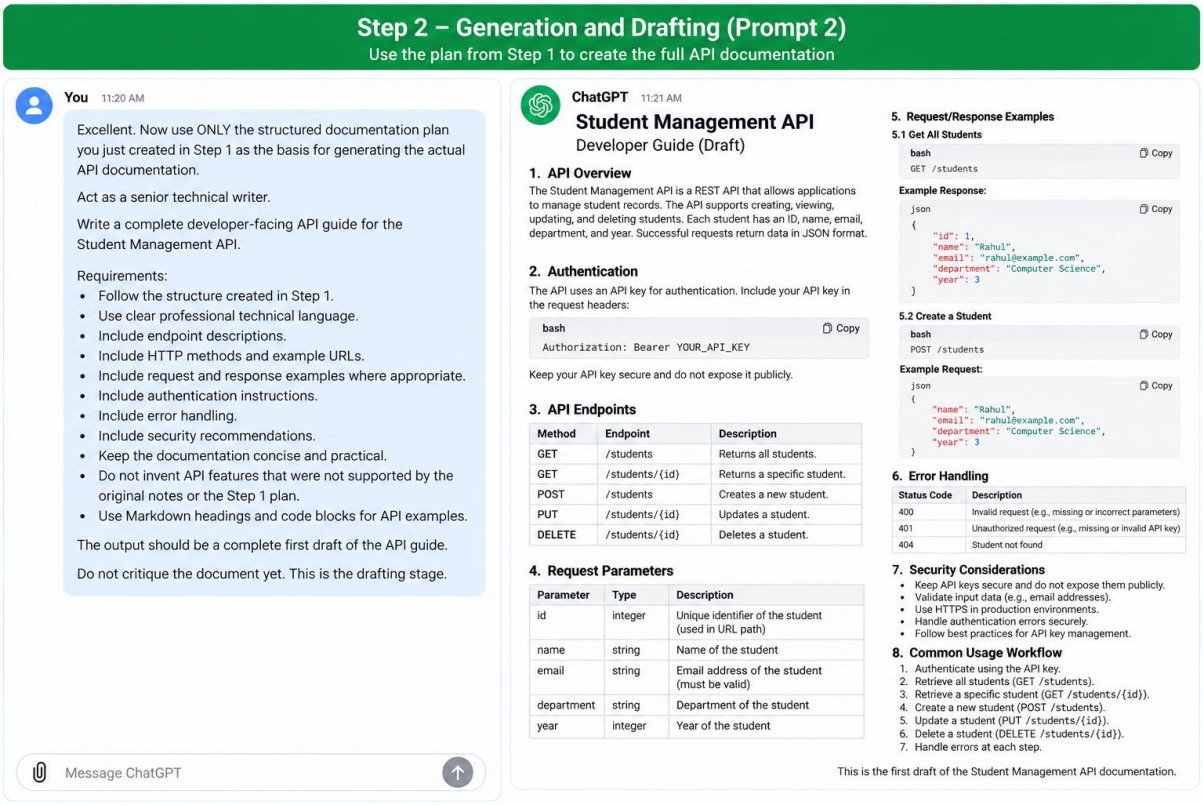
Write a complete developer-facing API guide for the Student Management API.

Requirements:

- Follow the structure created in Step 1.
- Use clear professional technical language.
- Include endpoint descriptions.
- Include HTTP methods and example URLs.
- Include request and response examples where appropriate.
- Include authentication instructions.
- Include error handling.
- Include security recommendations.
- Keep the documentation concise and practical.
- Do not invent API features that were not supported by the original notes or the Step 1 plan.
- Use Markdown headings and code blocks for API examples.

The output should be a complete first draft of the API guide.

Do not critique the document yet. This is the drafting stage.




## 7. STEP 2 — AI RESPONSE

**Student Management API**

**Developer Guide — Draft**

### 1. API Overview

The Student Management API is a REST API that allows applications to manage student records.

The API supports creating, viewing, updating, and deleting student information.

Each student record contains:

- ID
- Name
- Email
- Department
- Year

Successful requests return JSON data.

### 2. Authentication

The Student Management API uses an API key for authentication.

Applications should provide the required API key when making authenticated requests.

API keys are sensitive credentials and should be kept private.

Applications should never expose API keys publicly.

**3. API Endpoints**

| Method | Endpoint | Description |
|---|---|---|
| GET | /students | Returns all students |
| GET | /students/{id} | Returns a specific student |
| POST | /students | Creates a new student |
| PUT | /students/{id} | Updates a student |
| DELETE | /students/{id} | Deletes a student |

**4. Request Parameters**

Student records contain the following information:

| Parameter | Description |
|---|---|
| ID | Unique identifier of the student |
| Name | Name of the student |
| Email | Email address of the student |
| Department | Department of the student |
| Year | Academic year of the student |

The API should validate email addresses before accepting student information.

**5. Request and Response Examples**

**Get All Students**

**Method:** GET

**Endpoint:**

GET /students

**Example Response:**

```
[
  {
    "id": 1,
    "name": "Rahul",
```

```
    "email": "rahul@example.com",
    "department": "Computer Science",
    "year": 3
  }
]
```


**Get a Specific Student**

**Endpoint:**

GET /students/1

**Example Response:**


```
{
  "id": 1,
  "name": "Rahul",
  "email": "rahul@example.com",
  "department": "Computer Science",
  "year": 3
}
```


**Create a Student**

**Method:** POST

**Endpoint:**

POST /students

**Example Request:**


```
{
  "name": "Rahul",
  "email": "rahul@example.com",
  "department": "Computer Science",
  "year": 3
}
```


**Update a Student**

**Method:** PUT

**Endpoint:**

PUT /students/1

**Example Request:**

{
"name": "Rahul Kumar",
"email": "rahul@example.com",
"department": "Computer Science",
"year": 4
}

**Delete a Student**

**Method:** DELETE

**Endpoint:**

DELETE /students/1

**6. Error Handling**

The API may return the following HTTP status codes:

| Status Code | Meaning |
|---|---|
| 400 | Invalid request |
| 401 | Unauthorized request |
| 404 | Student does not exist |

Applications should handle API errors gracefully.

**7. Security Considerations**

API keys must be protected and should never be exposed publicly.

Applications should also validate input information, including email addresses.

Clients should safely handle authentication failures and other API errors.

**8. Common Usage Workflow**

1. Authenticate using the API key.
2. Retrieve student records.
3. Retrieve an individual student when required.
4. Create a new student.
5. Update an existing student.
6. Delete a student when required.
7. Handle errors appropriately.

**INSERT IMAGE HERE**

**Figure 2: Step 2 – Generation and Drafting showing Prompt 2 using the Step 1 plan to generate the API documentation draft.**

## 8. STEP 3 — EVALUATION AND REFINEMENT GATE

### 8.1 Purpose

The purpose of Step 3 is to evaluate the documentation draft against a specific checklist.

The AI must first identify problems or missing information and then produce the corrected final documentation.

This is the **critique and refinement gate** of the workflow.

### 8.2 Prompt 3

Before finalizing the API guide, act as a strict senior technical documentation reviewer.

Review the API guide you just created against this checklist:

1. Does the guide clearly explain the purpose of the API?
2. Is authentication explained correctly?
3. Are all five documented endpoints covered?
    - GET /students
    - GET /students/{id}
    - POST /students
    - PUT /students/{id}
    - DELETE /students/{id}
4. Are request and response examples understandable?
5. Are the 400, 401, and 404 errors explained?
6. Is email validation mentioned?
7. Are API-key security considerations included?
8. Is the documentation logically organized?
9. Did the draft introduce unsupported features or information?
10. Is the writing clear enough for a developer to use?

First, provide a concise bullet-point critique identifying any problems or missing information.

Second, fix every issue you identified.

Third, output the FINAL polished API documentation.

Clearly separate:

A. Review

B. Final Documentation

Do not merely describe the corrections. Actually rewrite the affected sections in the final documentation.

## 9. STEP 3 — AI REVIEW OUTPUT

### A. Review

The previous API documentation draft was evaluated against the required checklist.

- The purpose of the API is clearly explained.
- API-key authentication is included.
- All five required endpoints are covered.
- Request and response examples are provided.
- HTTP 400, 401, and 404 errors are explained.
- Email validation is mentioned.
- API-key security considerations are included.
- The documentation is logically organized.
- No major unsupported API features have been introduced.
- The writing is clear and suitable for developers.

### Overall Assessment

The draft satisfies the required checklist. Minor improvements are made to improve consistency, clarity, endpoint descriptions, and organization.

## 10. FINAL POLISHED API DOCUMENTATION

### Student Management API

### Developer Guide — Final Version

### 1. API Overview

The Student Management API is a REST API that allows applications to manage student records.

The API supports:

- Creating student records
- Viewing student records
- Updating student records
- Deleting student records

Each student record contains:

- ID
- Name
- Email
- Department
- Year

Successful requests return JSON data.

**2. Authentication**

The Student Management API uses an API key for authentication.

Applications must provide the appropriate API key when making authenticated requests.

API keys are sensitive credentials and must be kept private.

**Security requirement:** API keys should never be exposed publicly.

**3. API Endpoints**

**GET /students**

Returns all student records.

**Method:** GET

**Endpoint:**

GET /students

**Example Response:**


```
[
  {
    "id": 1,
    "name": "Rahul",
    "email": "rahul@example.com",
    "department": "Computer Science",
    "year": 3
  }
]
```


**GET /students/{id}**

Returns the student associated with the specified ID.

**Method:** GET

**Example:**

GET /students/1

**Example Response:**

```
{
  "id": 1,
  "name": "Rahul",
  "email": "rahul@example.com",
  "department": "Computer Science",
  "year": 3
}
```

If the specified student does not exist, the API returns HTTP 404.

**POST /students**

Creates a new student record.

**Method:** POST

**Endpoint:**

POST /students

**Example Request:**

```
{
  "name": "Rahul",
  "email": "rahul@example.com",
  "department": "Computer Science",
  "year": 3
}
```

The API should validate the email address before accepting the student record.

**PUT /students/{id}**

Updates the student associated with the specified ID.

**Method:** PUT

**Example:**

PUT /students/1

**Example Request:**

```
{
  "name": "Rahul Kumar",
  "email": "rahul@example.com",
  "department": "Computer Science",
  "year": 4
}
```

**DELETE /students/{id}**

Deletes the student associated with the specified ID.

**Method:** DELETE

**Example:**

DELETE /students/1

## 11. REQUEST PARAMETERS

| Parameter | Description |
|---|---|
| ID | Unique identifier of the student |
| Name | Name of the student |
| Email | Email address of the student |
| Department | Department of the student |
| Year | Academic year of the student |

Email addresses should be validated before student data is accepted.

## 12. ERROR HANDLING

The API may return the following HTTP status codes:

| Status Code | Description |
|---|---|
| 400 | Invalid request |
| 401 | Unauthorized request |
| 404 | Student does not exist |

Applications should handle these errors gracefully and provide appropriate feedback.

## 13. SECURITY CONSIDERATIONS

Applications using the API should follow these security practices:

- Keep API keys private.
- Never expose API keys publicly.
- Validate input data such as email addresses.
- Handle unauthorized requests safely.
- Handle API errors gracefully.

## 14. COMMON USAGE WORKFLOW

A typical application workflow is:

1. Authenticate using the API key.
2. Retrieve all students using GET /students.
3. Retrieve an individual student using GET /students/{id}.
4. Create a student using POST /students.
5. Update a student using PUT /students/{id}.
6. Delete a student using DELETE /students/{id}.
7. Handle HTTP 400, 401, and 404 errors appropriately.

**INSERT IMAGE HERE**

**Figure 3: Step 3 – Evaluation and Refinement showing the review checklist, critique, and final polished API documentation.**

## 15. WORKFLOW DEPENDENCY

The workflow contains a clear dependency between each stage.

**Step 1**

**Raw API Notes**

↓

**Structured Documentation Plan**

**Step 2**

**Structured Documentation Plan**

↓

**API Documentation Draft**

**Step 3**

**API Documentation Draft**

↓

**Evaluation and Critique**

↓

**Final Polished API Guide**

This demonstrates that each prompt builds on the output of the previous prompt.

## 16. MERMAID WORKFLOW DIAGRAM

```
flowchart TD
    A[Raw API Notes] --> B[Step 1: Extraction & Planning]
    B --> C[Structured Plan]
    C --> D[Step 2: Generation & Drafting]
    D --> E[API Documentation Draft]
    E --> F[Step 3: Evaluation & Refinement]
    F --> G[Final Polished API Guide]
```

**INSERT MERMAID FLOW IMAGE HERE**

**Figure 4: Multi-Step Prompt Workflow — Raw Notes to Final Polished API Guide.**

## 17. PROMPT CHAIN SUMMARY

| Step | Purpose | Input | Output |
|---|---|---|---|
| Step 1 | Extraction & Planning | Raw API notes | Structured documentation plan |
| Step 2 | Generation & Drafting | Step 1 plan | API documentation draft |
| Step 3 | Evaluation & Refinement | Step 2 draft | Critique + final API guide |

## 18. ASSESSMENT REQUIREMENT MAPPING

**Requirement 1 — Three Chained Prompts**

The workflow contains three distinct prompts executed sequentially.

**Prompt 1 → Prompt 2 → Prompt 3**

**Requirement 2 — Chaining Dependency**

Prompt 2 explicitly uses the structured plan created in Prompt 1.

Prompt 3 explicitly reviews the API documentation created in Prompt 2.

**Requirement 3 — Critique Gate**

Prompt 3 instructs the AI to evaluate the draft against ten specific criteria and then produce a corrected final version.

**Requirement 4 — Live Execution**

The three prompts should be executed in the assigned AI interface within the same chat session.

**Requirement 5 — Documentation**

The final submission contains the exact prompts, intermediate responses, final output, workflow diagram, and screenshot evidence.

## 19. SUBMISSION EVIDENCE

The following evidence should be included with the submission:

**Evidence 1**

Screenshot showing:

**Prompt 1 + Step 1 AI Response**

**Evidence 2**

Screenshot showing:

**Prompt 2 + Step 2 AI Response**

**Evidence 3**

Screenshot showing:

**Prompt 3 + Critique + Final Documentation**

**Evidence 4**

Mermaid workflow diagram.

**Evidence 5**

Live chat link, if available.

The assessment specifically requires the exact prompt chain, intermediate AI responses, and direct evidence of the live conversation thread.

## 20. CONCLUSION

This mini-assessment demonstrates how a complex technical documentation task can be divided into multiple connected AI stages.

In Step 1, raw project information is extracted and organized into a structured documentation plan.

In Step 2, the structured plan is transformed into a complete API documentation draft.

In Step 3, the draft is evaluated against a defined checklist and refined into the final polished API guide.

The final workflow demonstrates:

- Multi-step prompt chaining
- Output dependency
- Information extraction
- Structured planning
- Controlled generation
- Self-evaluation
- Critique and refinement
- Final documentation generation

**Final Workflow**

**RAW NOTES**

↓

**STEP 1 — EXTRACTION & PLANNING**

↓

**STRUCTURED PLAN**

↓

**STEP 2 — GENERATION & DRAFTING**

↓

**API DOCUMENTATION DRAFT**

↓

**STEP 3 — EVALUATION & REFINEMENT**

↓

**FINAL POLISHED API GUIDE**